Control Flow

Lecture 2 · Learning Python at the University of Glasgow

Khiem Nguyen

9 October 2026

Contents

1 Statements and code blocks **2**
1.1 Statements . . . 2
1.2 Compound statements and indentation . . . 2

2 What is control flow? **3**

3 Conditional statements **4**
3.1 Syntax . . . 4
3.2 Examples . . . 5
3.3 Nested `if-else` versus `if-elif-else` . . . 6
3.4 Combining conditions . . . 8
3.5 The `match-case` statement . . . 9
3.6 The `pass` statement . . . 10

4 Loops **10**
4.1 Two kinds of loops . . . 10
4.2 The `while` loop . . . 11
4.3 The `for` loop . . . 12
4.4 The `range()` function . . . 12
4.5 Looping over a list . . . 13
4.6 Nested loops . . . 14
4.7 Exiting a loop with `break` and skipping an iteration with `continue` . . . 14

5 Worked example: drawing a rectangle **19**

Summary **21**

Exercises **21**
Exercise 1: Grade conversion . . . 21
Exercise 2: A game menu with `while True` and `break` . . . 22
Exercise 3: Is the string numeric? . . . 23
Exercise 4: The multiplication table in style . . . 23
Exercise 5: Sum, maximum and minimum of random integers . . . 24

Further reading **25**

Learning objectives

By the end of this lecture, you should be able to:

- explain what statements, code blocks and indentation are in Python;
- make decisions in a program with `if`, `if-else`, `if-elif-else` and `match-case`;
- repeat work with `while` and `for` loops, and generate sequences of integers with `range()`;
- control loops with `break` and `continue`, and use `pass` as a placeholder;
- design a small program from pseudo-code and improve it step by step (refactoring).

1 Statements and code blocks

1.1 Statements

We have been using statements all along; now we give them a name.

- A **statement** is a single unit of instruction.
- A program contains one or more statements, each normally ending at the end of a line. (You *can* separate statements on one line with a semicolon `;`, but please don't.)

There are many kinds of statements, for example:

- assignment statements;
- decision (selection) statements;
- repetition (loop) statements;
- function calls;
- and so on.

1.2 Compound statements and indentation

In Python, a **compound statement** groups several statements into a single unit, called a **block**, by their level of **indentation**.

Indentation is essential in Python because the interpreter treats it as part of the language syntax. This is different from many other programming languages: C/C++ marks a block with curly braces `{...}`, and MATLAB closes a block with the keyword `end`; both ignore indentation.

Consider the following piece of code:

```
while student <= 10:      # while the student counter is at most 10
    result = input("Enter 1 (pass) or 2 (fail): ")
    result = int(result)  # the next exam result
```

```
    if result == 1:       # if the student passed
        passes += 1       # add one to passes
        print("Good news! One more student passed.")
    else:                 # otherwise
        failures += 1     # add one to failures
        print("Bad news! One more student failed.")
    student += 1          # add one to the counter
```

All the indented statements below `while student <= 10:` form a block that belongs to the `while` statement. Inside it, the two statements indented under `if result == 1:` form another block, which belongs to the `if` statement, and the two statements indented under `else:` form a third block, which belongs to the `else` clause. The final statement `student += 1` is back at the level of the `while` block, so it is executed in every repetition, whatever the exam result.

> Indent with four spaces
>
> By convention (PEP 8), each level of indentation is **four spaces**. Jupyter and most editors insert them automatically when you press **Tab** or **Enter** after a colon. Forgetting to indent is a syntax error:

```
if 3 > 2:
print("Three is greater than two")
```

```
IndentationError: expected an indented block after 'if' statement on line 1
 ↪  (1709809986.py, line 2)
  Cell In[19], line 2
    print("Three is greater than two")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

2 What is control flow?

A program's **control flow** is the order in which its statements are executed. In Python, the control flow is regulated by conditional statements, loops and function calls. Python uses the flow-control statements familiar from other programming languages, with a few twists of its own.

Every algorithm can be designed and implemented with just three types of control structure:

1. **Sequential statements.** As the name says, the statements are executed one after another, from top to bottom. This is what we did in Lecture 1. On its own, however, a sequence cannot react to circumstances: it always does exactly the same thing, whatever the data.

2. **Conditional statements.** These let a program test one or more conditions and execute different instructions depending on which condition is true. They branch the control flow so that the program behaves correctly in every situation it may encounter.
3. **Loop statements.** These repeat a block of instructions. This is what makes computers so powerful: they can carry out a huge number of repeated operations — far more than any human could — to reach the final result.

The Python statements that regulate control flow are:

1. `if`, `if-else` and `if-elif-else` statements, and `match-case` statements, for **conditional** flow;
2. `while` and `for` statements, together with the `range()` function, for **loops**;
3. the keywords `break` and `continue`, which, in combination with loops, *exit* the loop or *skip* the rest of the current iteration, respectively;
4. the `pass` statement, a placeholder that does nothing.

3 Conditional statements

As the English words suggest, conditional statements are quite natural to read. Right after the keyword `if`, a condition is tested to see whether it is `True` or `False`. If it is `True`, the block of statements after the `if` line is executed. In an `if-else` statement, if the condition is `False`, the block after `else` is executed instead. The `if-elif-else` statement is a compact way of writing several nested `if-else` statements.

3.1 Syntax

The `if` statement

```
if <expression>:        # note the colon ":"
    <statement>         # note the indentation
    ...
    <statement>
<outer-statement>       # note the end of the indentation
```

If `<expression>` evaluates to `True`, the indented block after the colon `:` is executed. If it evaluates to `False`, the block is skipped, and execution continues after it — in this case with `<outer-statement>`.

The `if-else` statement

```
if <expression>:
    <statement>
    ...
```

```
else:
    <statement>
    ...
```

If `<expression>` evaluates to `False`, the block after `else:` is executed instead.

The `if-elif-else` statement

```
if <expression-1>:
    <statement>
    ...
elif <expression-2>:
    <statement>
    ...
else:
    <statement>
    ...
```

The conditions are tested from top to bottom, and **only the block of the first condition that is `True` is executed**. If none of them is `True`, the `else` block is executed. You may have as many `elif` clauses as you like, and the `else` clause is optional.

> **Brackets are optional**
>
> You may write the condition in brackets, `if (x > 0):`, but this is unnecessary in Python, and the more common style is simply `if x > 0:`.

3.2 Examples

```
earth = 'round'
if earth == 'round':                # note the colon ":"
    print("The Earth is round.")    # note the indentation

if earth != 'round':
    print("The Earth is not round.")
else:                               # note the colon ":"
    print("The Earth is round.")    # note the indentation
```

```
The Earth is round.
The Earth is round.
```

The quadratic equation revisited. In Lecture 1, we computed the roots of the quadratic equation

$$c_2 x^2 + c_1 x + c_0 = 0.$$

The equation has real roots if and only if the discriminant is non-negative: $\Delta = c_1^2 - 4c_2c_0 \geq 0$. We should therefore test this condition *before* computing $\sqrt{\Delta}$.

```
from math import sqrt

# Coefficients of c2*x^2 + c1*x + c0 = 0.
# To make the program interactive, use e.g. c0 = float(input("Coefficient
 ↪  of x^0: ")).
c0, c1, c2 = 2.0, -3.0, 1.0

discriminant = c1**2 - 4*c2*c0
denominator = 2*c2
if discriminant < 0:                          # note the colon ":", not
 ↪  "then"
    print("The equation has no real roots.")  # note the indentation
else:                                         # note the colon ":"
    sqrt_discriminant = sqrt(discriminant)
    root_1 = (-c1 + sqrt_discriminant) / denominator
    root_2 = (-c1 - sqrt_discriminant) / denominator
    print("The two roots are:\n r1 = {0}\n r2 = {1}".format(root_1,
     ↪  root_2))
```

```
The two roots are:
 r1 = 2.0
 r2 = 1.0
```

> **Try it yourself**
>
> Change the coefficients to `c0, c1, c2 = 5.0, 2.0, 1.0` and run the cell again to see the other branch. Note that the final `print()` must be *inside* the `else` block: if it were outside, the program would try to print roots that were never computed.

3.3 Nested `if-else` versus `if-elif-else`

As mentioned above, the `if-elif-else` statement is a neat way of writing several nested `if-else` statements. The following two cells convert a mark into a letter grade and produce the same result, but the second version is much easier to read.

```
# Nested if-else statements
marks = 70
if marks <= 50:
    grade = 'D'
else:
    if marks <= 59:
        grade = 'C'
    else:
        if marks <= 69:
            grade = 'B'
        else:
            grade = 'A'
print("Your letter grade is", grade)
```

```
Your letter grade is A
```

```
# The same logic with if-elif-else
marks = 70
if marks <= 50:
    grade = 'D'
elif marks <= 59:
    grade = 'C'
elif marks <= 69:
    grade = 'B'
else:
    grade = 'A'
print("Your letter grade is", grade)
```

```
Your letter grade is A
```

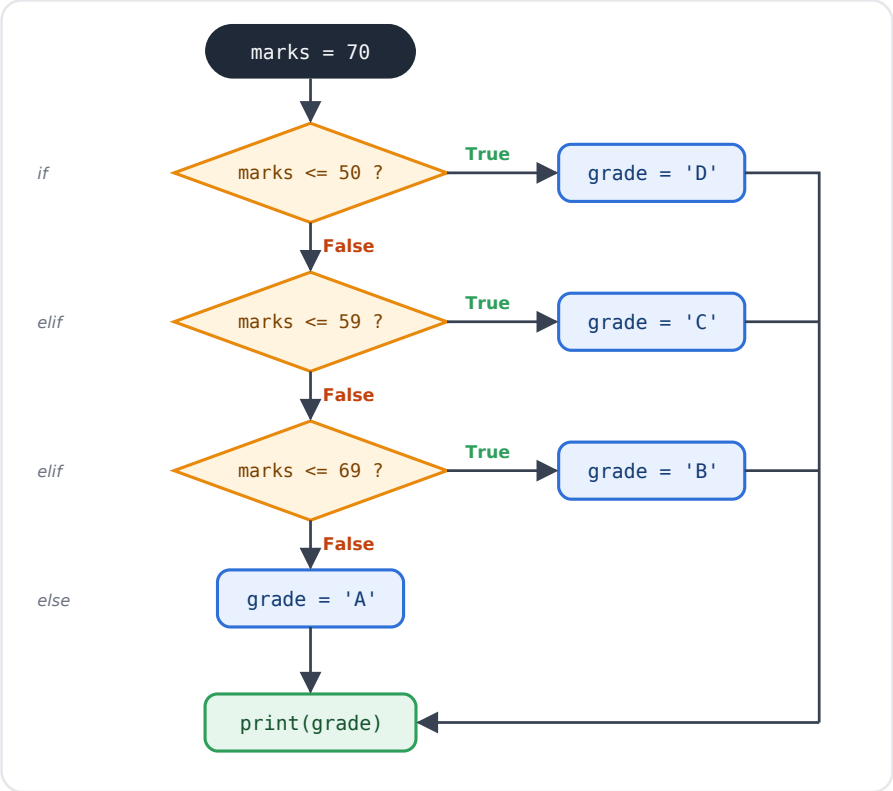


Figure 1: Flowchart of the `if-elif-else` statement above. The conditions are tested from top to bottom, and only the first branch whose condition is `True` is executed.

3.4 Combining conditions

Conditions can be combined with the logical operators `and`, `or` and `not` from Lecture 1. Python also allows **chained comparisons** such as `0 <= marks <= 100`, which means exactly the same as `0 <= marks and marks <= 100`, just as in mathematics.

```
marks = 72
attendance = 0.9    # fraction of classes attended

if 0 <= marks <= 100:                    # a chained comparison
    print("The mark is valid.")
if marks >= 70 and attendance >= 0.8:    # both conditions must be True
    print("Excellent work and attendance!")
if not attendance >= 0.8:
    print("Please come to class more often.")
```

```
The mark is valid.
Excellent work and attendance!
```

3.5 The `match-case` statement

Since Python 3.10, the `match-case` statement allows **pattern matching**. In its basic form, it looks very much like an `if-elif-else` statement that compares one value against a finite set of possible choices.

```
match <variable>:
    case <pattern-1>:
        <statement>
        ...
    case <pattern-2>:
        <statement>
        ...
    case _:
        <statement>
        ...
```

If the value of `<variable>` matches one of the patterns `<pattern-1>`, `<pattern-2>`, etc., the block belonging to the *first* matching pattern is executed. The special pattern `_` (an underscore) is a **wildcard** that matches anything, so `case _:` catches every value not matched before it. It is optional: if you do not need to catch the unmatched cases, leave it out. Several alternatives can be combined in one pattern with `|`, which means "or".

```
# Matching numbers
mark = 7
match mark:
    case 10:
        print("Grade = A+")
    case 9:
        print("Grade = A")
    case 8:
        print("Grade = A-")
    case _:                          # the wildcard matches anything else
        print("I don't care about your grade any more!")

# Matching strings, with alternatives separated by |
command = "E"
match command:
    case "e" | "E":
        print("Exit")
    case "h" | "H":
        print("Help")
```

```
    case _:
        print("Unknown command")
```

```
I don't care about your grade any more!
Exit
```

> **Optional topic**
>
> `match-case` is a recent addition to Python, and everything it does here can also be done with `if-elif-else`. It can do much more (matching the *structure* of data), but that is beyond this course. You may safely skip this section.

3.6 The `pass` statement

A block of code can never be empty: `if`, `while`, `for`, function and class definitions all require at least one statement. When you want to leave a block empty for now — for instance, to fill it in later — use the `pass` statement, which simply does nothing.

```
temperature = 25
if temperature > 30:
    pass    # TODO: switch on the air conditioning (to be written later)
else:
    print("The temperature is fine.")
```

```
The temperature is fine.
```

4 Loops

4.1 Two kinds of loops

There are broadly two kinds of loops in programming:

- **sentinel loops**, which run until some condition is met, where you do not know in advance when that will happen — for example, because it depends on user input or on the result of a complex calculation;
- **counter loops**, where you know in advance how many iterations will take place.

Python has two loop statements:

1. the `while` loop, which naturally implements **sentinel** loops;
2. the `for` loop, which naturally implements **counter** loops.

In practice, the two are interchangeable. A `while` loop can count, by using a counter variable in its condition; and, with the `break` statement, a `for` loop can stop early when a condition involving a complex expression or user input becomes true.

4.2 The `while` loop

Syntax

```
while <expression>:  # a colon right after the expression; brackets are not
 ↪  needed
    <statement-1>
    <statement-2>
    ...
    <statement-n>
```

- Python evaluates `<expression>`. If it is `False`, the loop ends, and execution continues after the loop.
- If it is `True`, the statements in the block are executed, and then Python goes back to evaluate `<expression>` again.

```
counter, value = 0, 1    # multiple assignment
n = 5                    # number of doublings; try n = int(input("n = "))
 ↪  instead
print("2^{0:<2} = {1}".format(counter, value))
while counter < n:
    value *= 2
    counter += 1
    print("2^{0:<2} = {1}".format(counter, value))
```

```
2^0  = 1
2^1  = 2
2^2  = 4
2^3  = 8
2^4  = 16
2^5  = 32
```

> ⚠ Infinite loops
>
> If the condition of a `while` loop never becomes `False`, the loop runs forever. For example, if we forgot the line `counter += 1` above, `counter < n` would always be `True`. In Jupyter, you can stop a running cell with **Kernel** > **Interrupt** (or the square *stop* button in the toolbar).

4.3 The `for` loop

The `for` statement in Python differs a little from what you may know from C or Pascal: instead of counting, it iterates over the items of any sequence (or, more generally, any **iterable**), in the order in which they appear.

Syntax

```
for element in iterable:
    <statement-1>
    <statement-2>
    ...
    <statement-n>
```

In the line `for element in iterable:`, the variable `element` takes the value of each item of `iterable` in turn, and the block is executed once for each of them. The iterable may be a string, a list, a `range()` and many other data types. Iteration is a very important concept in modern programming: you can picture an iterable as a queue at a service counter, where every customer is served in order, one at a time.

```
for letter in "Glasgow":     # a string is a sequence of characters
    print(letter, end=" ")
```

```
G l a s g o w
```

4.4 The `range()` function

To repeat something a given number of times, we combine `for` with the built-in function `range()`, which produces a sequence of integers:

Call	Produces
`range(stop)`	`0, 1, 2, ..., stop - 1`
`range(start, stop)`	`start, start + 1, ..., stop - 1`
`range(start, stop, step)`	`start, start + step, start + 2*step, ...` (before `stop`)

Note that `stop` itself is **never** included. In the next cell, we wrap `range()` in `list()` only to display all the numbers at once; lists are the topic of Lecture 3.

```
print(list(range(5)))          # 0 to 4: stop = 5 is excluded
print(list(range(2, 8)))       # 2 to 7
print(list(range(1, 10, 2)))   # the odd numbers from 1 to 9
```

```
print(list(range(10, 0, -3)))  # counting down: 10, 7, 4, 1
```

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[1, 3, 5, 7, 9]
[10, 7, 4, 1]
```

Here are the powers of two again, this time with a `for` loop. Compare it with the `while` version above: there is no counter to initialise or update, so there is no risk of an infinite loop.

```
value = 1
n = 5
print("2^{0:<2} = {1}".format(0, value))
for i in range(n):
    value *= 2
    print("2^{0:<2} = {1}".format(i + 1, value))
```

```
2^0  = 1
2^1  = 2
2^2  = 4
2^3  = 8
2^4  = 16
2^5  = 32
```

4.5 Looping over a list

Even better, the sequence can be defined by the data themselves rather than by a counting variable. Here is a first look at a **list**: a sequence of values written between square brackets. We study lists properly in Lecture 3.

```
my_list = ["I", "have", 2, "dogs", "and", 3, "cats"]

# First loop: print each element on its own line
for x in my_list:
    print(x)

# Second loop: print the elements on one line, separated by spaces
for x in my_list:
    print(x, end=' ')
```

```
I
have
2
```

```
dogs
and
3
cats
I have 2 dogs and 3 cats
```

4.6 Nested loops

A loop can contain another loop. For each single iteration of the **outer** loop, the **inner** loop runs completely.

```
x = 1
while x < 4:
    y = x
    while y < 4:
        y += 1
        print("\t inner loop: y =", y)
    x += 1
    print("outer loop: x =", x)
```

```
     inner loop: y = 2
     inner loop: y = 3
     inner loop: y = 4
outer loop: x = 2
     inner loop: y = 3
     inner loop: y = 4
outer loop: x = 3
     inner loop: y = 4
outer loop: x = 4
```

4.7 Exiting a loop with `break` and skipping an iteration with `continue`

`break` and `continue` are two important keywords that are used inside loops. Their names describe what they do:

Keyword	What it does
`break`	stops executing the rest of the block of the **innermost** loop and ***exits that loop*** immediately
`continue`	stops executing the rest of the block of the **innermost** loop and ***continues with the next iteration***

Consider the following outline, with two nested `while` loops:

```
while <outer-condition>:
    <statement-outer-1>
    while <inner-condition>:
        if <condition>:
            <statement-inner-1>
            break                # leave the INNER loop immediately ...
        <statement-inner-2>
    <statement-outer-2>          # ... and continue from here
```

If `<condition>` is `True`, `<statement-inner-1>` is executed, and then `break` is reached. The inner loop ends at once, and execution continues with the first statement after it, `<statement-outer-2>`, which is still inside the outer loop. In other words, `break` only leaves the loop that it belongs to most directly. Any statements written after `break` in the same block would never be executed, so do not write any.

Now replace `break` with `continue`:

```
while <outer-condition>:
    <statement-outer-1>
    while <inner-condition>:
        if <condition>:
            <statement-inner-1>
            continue             # skip the rest of THIS iteration ...
        <statement-inner-2>      # ... so this line is not executed this
        ↪ time
    <statement-outer-2>
```

If `<condition>` is `True`, `<statement-inner-1>` is executed, and then `continue` skips the rest of the current iteration of the inner loop, i.e. `<statement-inner-2>`. Unlike `break`, `continue` does **not** exit the inner loop: execution jumps back to the loop header, which re-tests `<inner-condition>` (a `for` loop would move on to its next element). The inner loop ends normally, only when `<inner-condition>` becomes `False`.

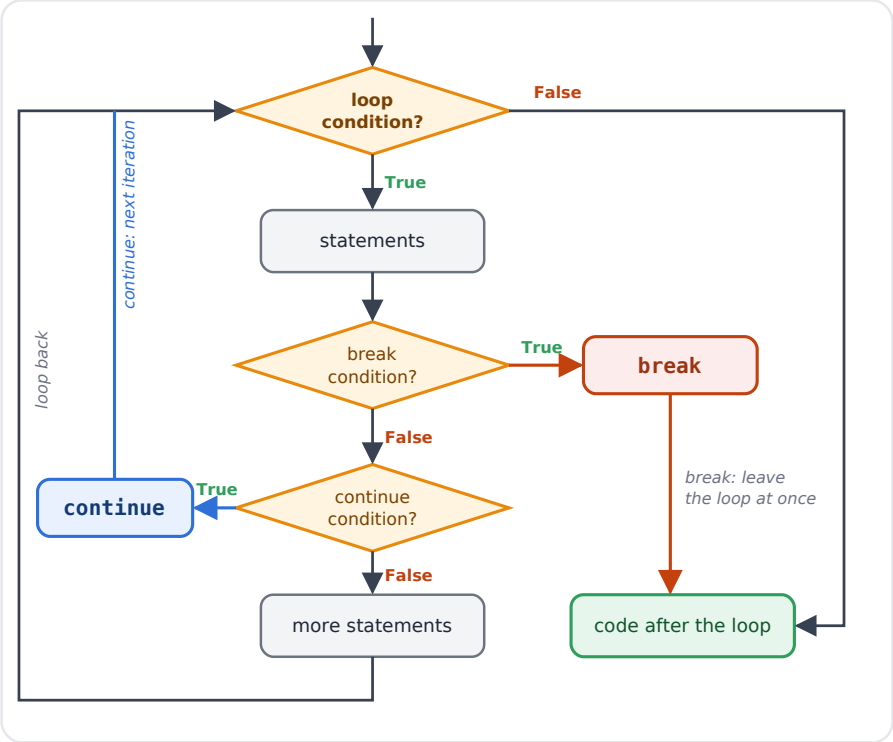


Figure 2: Control flow inside a loop: `continue` jumps back to the loop condition, whereas `break` leaves the loop immediately.

Examples. The first example needs no keyboard input:

```
for number in range(1, 20):
    if number % 2 == 1:
        continue          # skip the odd numbers ...
    if number > 10:
        break             # ... and stop the loop completely after 10
    print(number, end=" ")
```

```
2 4 6 8 10
```

The next examples are interactive. We want to leave a `while` loop as soon as the user types `10`. One way is to use a "flag" variable such as `keep_going`. This practice should be avoided where possible, for two reasons: (1) the code is longer, because we need an extra variable; and (2) with several nested loops, we would need several flag variables, `keep_going_1`, `keep_going_2`, and so on.

```
# With a flag variable (avoid this style)
keep_going = True
while keep_going:
```

```
    x = int(input("Type a number (10 to stop): "))
    if x == 10:
        print("Stop")
        keep_going = False
```

The `break` statement does the same job more neatly:

```
# With break (preferred)
while True:
    x = int(input("Type a number (10 to stop): "))
    if x == 10:
        print("Stop")
        break
```

In the next example, we have two `while True` loops, called the outer loop and the inner loop. To escape from the inner loop, the user must type `x`; to escape from the outer loop, the user must type `y`. As long as we do not leave the outer loop, it will enter the inner loop again. Therefore, the only way out of both loops is to type `x` (Enter) and then `y` (Enter); anything other than `y` after `x` sends us through the inner loop once more.

```
counter_for_outer_loop = 1
counter_for_inner_loop = 1
while True:
    counter_for_inner_loop = 1
    print("You are entering the INNER loop!")
    while True:
        key = input("\tType a key! The key 'x' exits the INNER loop: ")
        if key == 'x':
            print("\tOUT of the INNER loop. INNER iteration =",
             ↪  counter_for_inner_loop)
            break
        else:
            print("\tIN the INNER loop. INNER iteration =",
             ↪  counter_for_inner_loop)
        # This line is at the same indentation as if-else,
        # so it belongs to the block of the inner while loop.
        counter_for_inner_loop += 1

    print("You have left the INNER loop and are now in the OUTER loop!")
    key = input("Type a key! The key 'y' exits the OUTER loop: ")
    if key == 'y':
```

```
        print("OUT of the OUTER loop!")
        break
    else:
        print("You are still in the OUTER loop. OUTER iteration =",
        ↪ counter_for_outer_loop)
    # This line belongs to the block of the outer while loop.
    counter_for_outer_loop += 1

print("--------------------------\n")
print("         OUTER iteration =", counter_for_outer_loop)
print("The last INNER iteration =", counter_for_inner_loop)
```

Finally, we keep adding up the numbers typed by the user. If the input is not numeric, we skip the addition with `continue`; the user exits with `x`.

```
# do not call this variable "sum": that is the name of a built-in function!
total = 0
print("Total = {}".format(total))
print("Calculating the sum of the numbers you type. To exit, type 'x'!")
while True:
    x = input("Type a number: ")
    if x == 'x':
        print("You exited the loop with 'x'.")
        break
    elif not x.isnumeric():
        print("Input = {}, which is not numeric! The total is not
        ↪ updated.".format(x))
        continue

    total += int(x)
    print("Total <-- Total + {0} = {1}".format(x, total))

print("Final total = {}".format(total))
```

A sample run:

```
Total = 0
Calculating the sum of the numbers you type. To exit, type 'x'!
Type a number: 10
Total <-- Total + 10 = 10
Type a number: ten
```

```
Input = ten, which is not numeric! The total is not updated.
Type a number: 20
Total <-- Total + 20 = 30
Type a number: x
You exited the loop with 'x'.
Final total = 30
```

5 Worked example: drawing a rectangle

The task. Read two integers, `width` and `height` (denoted w and h), and draw a (rather useless) rectangle with these dimensions. For example, for $w = 12$ and $h = 5$, the drawing should look like this:

```
x----------x
|----------|
|----------|
|----------|
x----------x
```

Each line has 10 dashes and 2 end characters, making up the width $w = 12$. Similarly, there are three lines with vertical bars between two lines with corners, making up the height $h = 5$.

Pseudo-code. Before writing any code, we describe the algorithm in plain words:

1. Read w and h (or simply assign them).
2. Line 1: print `x`, then $w - 2$ dashes `-`, then `x`.
3. Lines 2 to $h - 1$: print `|`, then $w - 2$ dashes, then `|`.
4. Line h (the last line): repeat line 1.

Remember that `print()` ends with a new line by default; `print("something", end="")` prints "something" and nothing else.

A first solution. Translating the pseudo-code directly gives:

```
h = 5
w = 12
for i in range(h):
    j = i + 1                    # line number, counting from 1
    if j == 1:
        print("x", end="")
        for k in range(w - 2):
```

```
            print("-", end="")
        print("x")
    elif j > 1 and j < h:
        print("|", end="")
        for k in range(w - 2):
            print("-", end="")
        print("|")
    else:
        print("x", end="")
        for k in range(w - 2):
            print("-", end="")
        print("x")
```

```
x----------x
|----------|
|----------|
|----------|
x----------x
```

Refactoring 1. The first and last lines are identical, so we can merge their branches. **Refactoring** means improving the structure of code without changing what it does.

```
h = 5
w = 12
for i in range(h):
    j = i + 1
    if j == 1 or j == h:          # first or last line
        print("x", end="")
        for k in range(w - 2):
            print("-", end="")
        print("x")
    else:                         # all the lines in between
        print("|", end="")
        for k in range(w - 2):
            print("-", end="")
        print("|")
```

```
x----------x
|----------|
|----------|
|----------|
x----------x
```

Refactoring 2. Recall from Lecture 1 that a string can be repeated with `*`. This removes the inner loops altogether:

```
h = 5
w = 12
for j in range(1, h + 1):          # j = 1, 2, ..., h
    if j == 1 or j == h:
        print("x" + "-" * (w - 2) + "x")
    else:
        print("|" + "-" * (w - 2) + "|")
```

```
x----------x
|----------|
|----------|
|----------|
x----------x
```

In Lecture 4, we will refactor this program once more, using a *function*.

Summary

- A block of statements is defined by its **indentation** (four spaces per level); a colon `:` always introduces a new block.
- `if`, `if-else` and `if-elif-else` choose which block to execute; only the first branch whose condition is `True` runs. `match-case` compares one value against several patterns, with `case _:` as the catch-all.
- `while` repeats a block as long as a condition is `True`; `for` iterates over the items of a sequence, often a `range()`. `range(start, stop, step)` never includes `stop`.
- `break` leaves the innermost loop immediately; `continue` skips to its next iteration; `pass` does nothing and serves as a placeholder.
- Write pseudo-code first, get a working solution, then refactor it.

Exercises

Exercise 1: Grade conversion

USA	Grade point	Glasgow grade
A+	4.0	A1
A	4.0	A3
A-	3.67	A5

USA	Grade point	Glasgow grade
B+	3.33	B1
B	3.0	B2
B-	2.67	B3
C+	2.33	C1
C	2.0	C2
C-	1.67	C3
D+	1.33	D1
D	1.0	D2
D-	0.67	D3
F	0.0	E2

Write a program that receives a numeric grade point and outputs the corresponding USA letter grade as well as the Glasgow grade. Use the comparison lower $<$ grade $\leq$ higher, where the higher bound is inclusive ($\leq$) and the lower bound is exclusive ($<$).

> **Hint**
>
> You can certainly use an `if-elif-else` statement. However, if you find yourself copying and pasting the same statement for many conditions, it is probably time to think about a `for` loop over a list of thresholds and grades. A `for` loop is the natural choice here because the number of cases to test is finite and known in advance.

Exercise 2: A game menu with `while True` and `break`

Write a program that asks the user to choose one of five options:

- `1`: single player
- `2`: two players
- `3`: three players
- `4`: four players
- `e`: exit

Further requirements:

- For the four game options, only the numeric inputs 1 to 4 are accepted.
- For the exit option, both `e` and `E` are accepted.
- Any other input is invalid. In that case, print a message saying that the input is invalid and (if you like) remind the user of the valid inputs.
- The loop ends only when the user chooses `e`, `E`, 1, 2, 3 or 4. Print the choice.

> **Hint**
>
> Think of an *infinite* loop, `while True:`, and how to leave it with `break`. This is also a good opportunity to try `match-case`.

Exercise 3: Is the string numeric?

We often ask the user to type a number. However, `input()` always returns a string, which we then need to convert into a number. This can go wrong if the user types a non-numeric string: for example, by accident, the user types `123q` instead of `1231`, because the key `q` is next to the key `1`. We therefore want to check that the string really represents a number *before* converting it. Write a `while True` loop that asks the user to type a number and, if the input is not numeric, asks again until the user gets it right.

> **Hint**
>
> Use the string method `isnumeric()`: `my_string.isnumeric()` returns `True` if `my_string` consists only of digits.

Exercise 4: The multiplication table in style

We often want to print a list of results in an eye-pleasing format. In this exercise, print the multiplication table neatly in four columns, like this:

Column 1	Column 2	Column 3	Column 4
$2 \times 1 = 2$	$3 \times 1 = 3$	$4 \times 1 = 4$	$5 \times 1 = 5$
$2 \times 2 = 4$	$3 \times 2 = 6$	$4 \times 2 = 8$	$5 \times 2 = 10$
...	...	...	...
$2 \times 10 = 20$	$3 \times 10 = 30$	$4 \times 10 = 40$	$5 \times 10 = 50$

Then print an empty line, followed by the next block of tables:

Column 1	Column 2	Column 3	Column 4
$6 \times 1 = 6$	$7 \times 1 = 7$	$8 \times 1 = 8$	$9 \times 1 = 9$
$6 \times 2 = 12$	$7 \times 2 = 14$	$8 \times 2 = 16$	$9 \times 2 = 18$
...	...	...	...
$6 \times 10 = 60$	$7 \times 10 = 70$	$8 \times 10 = 80$	$9 \times 10 = 90$

So there are two blocks, each with four columns.

Special requirement: the equals signs = must be aligned in all eight columns.

> **Hint**
>
> Use string formatting for the alignment. In $a \times b$, the number b may be `10`, which has two digits; this makes the last lines, `2 x 10`, `3 x 10`, look out of line with all the lines above them. Imagine instead that the numbers `1`, `2`, ..., `10` are all aligned to the left and occupy the space of **two** characters rather than one. Then `1` and `10` take up the same width in the output. To align a value to the left in a field two characters wide, use
>
> ```
> print("{0:<2}".format(value), end="")
> ```
>
> where `0` refers to the first argument of `format()`, the colon `:` starts the format specification, and `<2` means "aligned to the left in a field two characters wide". Change the value of `end` to add a space after b. The details depend on your own experiments; see Lecture 1 for more about string formatting.

Exercise 5: Sum, maximum and minimum of random integers

Pseudo-random numbers. The function `randint(a, b)` from the module `random` returns a random integer N with $a \leq N \leq b$ (both end points included):

```
import random as rd
rd.randint(a, b)    # a and b must be defined beforehand, of course
```

The numbers are only *pseudo-random*: they are produced by a deterministic algorithm, starting from a value called the **seed**. By default, Python takes the seed from the operating system (or the current time), so you get different numbers every time you run the program. If you fix the seed with `rd.seed(0)`, you get exactly the same sequence of numbers every time, which makes it much easier to test your program.

Appending elements to a list. A list can start empty, `my_list = []`. To add the value `x` at the end of the list, use `my_list.append(x)` (or, less efficiently, `my_list = my_list + [x]`).

Your task. Create a list of $N = 20$ random integers between 1 and 100, then compute the sum of these integers and find the largest and the smallest of them — **without** using the built-in functions `sum()`, `max()` and `min()`. Use the seed zero, `rd.seed(0)`, so that your results do not change every time you re-run the program. (Afterwards, you may use `sum()`, `max()` and `min()` to check your answers.)

Hint

Use a variable `total` to accumulate the additions of all the numbers. (Do not call it `sum`: that would hide the built-in function!) Similarly, use variables `minimum` and `maximum` to *keep track* of the smallest and largest values seen so far. For example, start with a deliberately poor guess for `minimum` — a number larger than any number in the list, such as `float('inf')` — or simply with the first element. Then go through the elements one by one and update `minimum` whenever you find a smaller value. Do the same for `maximum`.

Further reading

- The official Python tutorial: More Control Flow Tools.
- PEP 636 — Structural Pattern Matching: Tutorial, for much more about `match-case`.